

关系代数 + SQL 综合 —— 练习题

本卷共 10 大题，按难度递增排序（基础 6 题 + 提高 4 题），总计 75 分。不计分、不限时，供集中突破使用。所有关系模式、查询场景均为原创。

- > 题目文件末尾有「答案（速查版）」用于自检；详细解题过程与金句汇总在配套解析文件中。
- > 关系代数符号约定： π 投影、 σ 选择、 $\bowtie$ 连接、 $\div$ 除、G 分组、F 函数（COUNT/SUM/AVG/MAX 等）。

板块一：关系代数·基础

关系模式（学生选课场景，第 1/4/5/6 题用）：

- 学生 S(学号, 姓名, 性别, 所在系, 入学年份)
- 课程 C(课程号, 课程名, 学分, 所属系)
- 选课 SC(学号, 课程号, 成绩, 学期)
- 教师 T(教师号, 姓名, 职称, 所在系)
- 任课 TC(教师号, 课程号, 学期, 教室)

> 关系模式（电商场景，第 2 题用）：

- 顾客 K(顾客号, 姓名, 城市, 注册日期)
- 商品 P(商品号, 商品名, 类别, 单价)
- 订单 H(订单号, 顾客号, 下单日期, 总金额)
- 订单明细 D(订单号, 商品号, 数量, 小计金额)

> 关系模式（银行场景，第 3 题用，与板块三第 9 题字段一致）：

- 客户 C(客户号, 姓名, 城市, 客户等级)
- 账户 A(账户号, 客户号, 开户行号, 余额, 开户日期)
- 交易记录 T(交易号, 账户号, 交易日期, 交易类型, 金额)（第 3 题未使用，仅作补充）

> 补充说明（学生选课场景）：第 4 题用到的「先修关系」隐含在 C 表的所属系/学分的语义之外——为简化，将「数据库」这门课的所有先修课视为已知常量集合（如：'数据结构'、'离散数学'、'操作系统'），用 σ 课程名 $\in$ {'数据结构','离散数学','操作系统'} (C) 表示先修课集合。

第 1 题（单表 π/σ 综合，5 分）

请用关系代数表达式完成以下查询（沿用板块一学生选课关系模式）：

（1）查询“计算机系”且“2020 年及以后入学”的所有“女生”学生的姓名与学号。（5 分）

要求：写出完整关系代数表达式，并指出哪些子表达式分别负责“过滤”“投影”。

第 2 题（多表 $\bowtie$ ，5 分）——电商场景

请用关系代数表达式完成以下查询（沿用板块一电商关系模式）：

（1）查询“北京”或“上海”顾客在 2024 年下过订单的所有商品名与单价（结果需去重——同一商品被多人下单只显示一次）。（5 分）

要求：写出完整关系代数表达式，并说明连接顺序与去重位置。

第3题 (G 分组 + F 聚合, 5 分) — 银行场景

请用关系代数表达式完成以下查询 (沿用板块一银行关系模式) :

(1) 查询每位客户的"客户号、姓名、账户总数、账户总余额" (即每位客户名下开了多少个账户、余额合计多少) 。 (5 分)

要求 : 写出完整关系代数表达式, 并标注 G 分组的依据与 F 聚合函数的对应关系。

第4题 (÷ 除运算, 10 分) — 学生选课场景

请用关系代数表达式完成以下查询 (沿用板块一学生选课关系模式, '数据库'课的先修课视为'数据结构'、'离散数学'、'操作系统'三门) :

(1) 查询"选修了'数据库'课所有先修课"的学生的学号与姓名 (按宽口径解读, 不要求该学生也选修'数据库'课本身) 。 (5 分)

(2) 查询"选修了所有课程"的学生的姓名。 (5 分)

要求 : 写出完整关系代数表达式, 并指出"÷ 除法"的被除关系与除关系分别是什么。

板块二 : 关系代数 + SQL 过渡

第5题 (关系代数综合 $\pi/\sigma/\bowtie$, 5 分) — 学生选课场景
请用关系代数表达式完成以下查询 (沿用板块一学生选课关系模式) :

(1) 查询"至少有一位'教授'任课"的所有课程的"课程号、课程名、任课教师姓名、教室"。 (5 分)

要求 : 写出完整关系代数表达式, 并说明 σ 选择条件与 $\bowtie$ 连接键的对应关系。

第6题 (SQL 基础 : SELECT/WHERE/ORDER BY/聚合, 5 分) — 学生选课场景

请用 SQL 语句完成以下查询 (沿用板块一学生选课关系模式) :

(1) 查询"计算机系"学生"2023 秋季学期" (即 SC.学期 = '2023秋') 所有课程的平均成绩, 按平均成绩降序排列, 列出姓名与平均成绩。 (5 分)

要求 : 写出完整 SQL 语句, 并标注 JOIN、WHERE、聚合、ORDER BY 各子句的作用。

板块三 : SQL· 提高

关系模式 (选课场景·简化, 第 7-8 题用) :

- 学生 S(学号, 姓名, 性别, 出生年份, 所在系)
- 课程 C(课程号, 课程名, 学分, 所属系)
- 选课 SC(学号, 课程号, 成绩)

>

关系模式 (银行场景·含支行, 第 9 题用) :

- 客户 C(客户号, 姓名, 城市, 客户等级)
- 账户 A(账户号, 客户号, 开户行号, 余额, 开户日期)
- 支行 B(支行号, 支行名, 城市)

>

关系模式 (航班场景, 第 10 题用) :



- 乘客 P(乘客号, 姓名, 身份证号, 联系电话)
- 航班 F(航班号, 出发机场, 到达机场, 起飞时间, 票价)
- 订票 B(乘客号, 航班号, 订票日期, 座位号, 票数)

第 7 题 (嵌套子查询 NOT EXISTS + IN, 10 分) — 选课场景

请用 SQL 语句完成以下查询 (沿用板块三选课·简化关系模式) :

- (1) 用 NOT EXISTS 双层嵌套实现: 查询"选修了'计算机系'开设的所有课程"的学生的姓名。(5 分)
- (2) 用 MIN/MAX 聚合 + IN 子查询实现: 查询"成绩高于'张三'所有课程成绩"的学生的姓名 (假设张三的学号已知、且其选课记录可能有多条; 本子查询不要求排除张三本人)。(5 分)

要求: 写出完整 SQL 语句; 对 (1) 需说明 NOT EXISTS 双层嵌套的"内层否定什么、外层否定什么"; 对 (2) 需说明 IN 子查询返回的集合含义。

第 8 题 (相关子查询 + NOT EXISTS 相关, 10 分) — 选课场景

请用 SQL 语句完成以下查询 (沿用板块三选课·简化关系模式) :

- (1) 查询"每门课程成绩都高于该课程平均成绩"的学生的学号。(5 分)
提示: 相关子查询的"相关"指子查询中引用外层查询的元组 (如 SC1.课程号 = SC2.课程号)。
- (2) 查询"没有挂过任何课程" (即所有课程成绩 ≥ 60) 的学生的姓名, 用 NOT EXISTS 相关子查询实现。(5 分)
提示: 等价表述为"不存在一门课使该学生成绩 < 60 "。

要求: 写出完整 SQL 语句; 标注相关子查询的"关联列"在哪个子句出现; 解释 NOT EXISTS 的"双重否定"逻辑。

第 9 题 (多表 JOIN + 聚合 + HAVING, 10 分) — 银行场景

请用 SQL 语句完成以下查询 (沿用板块三银行·含支行关系模式) :

- (1) 查询"账户总余额超过 10 万元"的客户的姓名与总余额 (按总余额降序)。(5 分)
- (2) 查询"在 ≥ 3 个不同支行都开过户"的客户的姓名、所在城市与开过账户的支行数 (按支行数降序)。(5 分)
提示: 注意对 (客户号, 开户行号) 去重后再统计。

要求: 写出完整 SQL 语句; 标注 JOIN 连接顺序、GROUP BY 分组键、HAVING 过滤条件三者如何配合。

第 10 题 (SQL 综合: LEFT JOIN + COALESCE + 聚合, 10 分) — 航班场景

请用 SQL 语句完成以下查询 (沿用板块三航班关系模式) :

- (1) 查询"至少订过 3 次票" (按航班号去重统计航班数) 的乘客的姓名与订票次数 (按订票次数降序; 次数相同时按姓名升序)。(5 分)
- (2) 查询"每位乘客累计购票支出", 用 LEFT JOIN 航班表 + COALESCE 处理未订票的乘客显示 0, 输出乘客号、姓名、累计金额 (未订票的乘客也要列出, 累计金额为 0)。(5 分)
提示: 累计金额 = SUM(票数 $\times$ 票价); LEFT JOIN 保证未订票乘客不丢失。

要求: 写出完整 SQL 语句; 标注 DISTINCT 的作用位置、LEFT JOIN 的 ON 子句完整性、COALESCE 的兜底逻辑。



答案（速查版）

速查版只给最终答案。详细推导、考点解读与金句见配套解析文件。

板块一：关系代数·基础

第1题：

- (1) π 姓名,学号 (σ 所在系='计算机系' $\wedge$ 性别='女' $\wedge$ 入学年份 ≥ 2020 (S))
- σ 所在系='计算机系' $\wedge$ 性别='女' $\wedge$ 入学年份 ≥ 2020 (S)：过滤（按三个条件合取筛选学生元组）
- π 姓名,学号 (...)：投影（从筛选后的元组取出"姓名"和"学号"两列）

第2题：

- (1) π 商品名,单价 (σ 城市 $\in\{\text{'北京','上海'}\} \wedge 2024-01-01 \leq \text{下单日期} \leq 2024-12-31$ ($K \bowtie H \bowtie D \bowtie P$)))
- 连接顺序： $K \bowtie H$ （顾客与订单） $\rightarrow \bowtie D$ （加订单明细） $\rightarrow \bowtie P$ （加商品）
- DISTINCT 去重：关系代数 π 本身是集合语义，自带去重（同一商品被多人下单只出现一次）；若翻译为 SQL 需显式加 DISTINCT：SELECT DISTINCT P.商品名, P.单价 ...

第3题：

- (1) π 客户号,姓名,账户总数,账户总余额 ($C \bowtie (G \text{ 客户号 } F \text{ COUNT(账户号) AS 账户总数, SUM(余额) AS 账户总余额 (A))$)
- G 分组键：客户号（每位客户一个组）
- F 聚合：COUNT(账户号) 统计账户数，SUM(余额) 累加余额

第4题：

- (1) π 学号,姓名 ($S \bowtie (\pi \text{ 学号,课程号 (SC)} \div \pi \text{ 课程号 } (\sigma \text{ 课程名} \in \{\text{'数据结构','离散数学','操作系统'}\} (C))))$)
- (2) π 姓名 ($S \bowtie (\pi \text{ 学号,课程号 (SC)} \div \pi \text{ 课程号 (C))$)

板块二：关系代数 + SQL 过渡

第5题：

- (1) 思路：先找出"有教授任教的课程号集合" ($\pi \text{ 课程号 } (\sigma \text{ 职称='教授' (TC} \bowtie T))$)，再用该集合 $\bowtie$ 所有任教信息（不限职称），得到完整任课清单
- 关系代数表达式：

```
 $\pi \text{ 课程号,课程名,姓名,教室 } ($   
   $(\pi \text{ 课程号 } (\sigma \text{ 职称='教授' (TC} \bowtie T)))$   
   $\bowtie \text{TC} \bowtie T \bowtie C$   
)
```

• 易错点：如果直接写 $\pi \text{ 课程号,课程名,姓名,教室 } (\sigma \text{ 职称='教授' (C} \bowtie \text{TC} \bowtie T))$ ，会把"教授+讲师混合任课"的课程中"讲师"那行过滤掉，导致输出不全。正确做法是"先筛课程号，再 $\bowtie$ 所有任课行"——这是" σ 在 $\bowtie$ 之前"原则的反例：不能让 σ 提前吃掉非目标行。

第6题：

- (1)

```
SELECT S.姓名, AVG(SC.成绩) AS 平均成绩  
FROM S JOIN SC ON S.学号 = SC.学号  
WHERE S.所在系 = '计算机系' AND SC.学期 = '2023秋'  
GROUP BY S.学号, S.姓名  
ORDER BY 平均成绩 DESC;
```

板块三：SQL·提高

第7题：

- (1) NOT EXISTS 双层嵌套：



```

SELECT 姓名 FROM S
WHERE NOT EXISTS (
  SELECT 1 FROM C
  WHERE C.所属系 = '计算机系'
  AND NOT EXISTS (
    SELECT 1 FROM SC
    WHERE SC.学号 = S.学号 AND SC.课程号 = C.课程号
  )
);

```

- (2) 改用 MIN/MAX 聚合 (推荐 , 比双重否定更直观) .

```

SELECT 姓名 FROM S
WHERE 姓名 <> '张三'
AND 学号 IN (
  SELECT 学号 FROM SC
  GROUP BY 学号
  HAVING MIN(成绩) > (
    SELECT MAX(SC2.成绩) FROM SC SC2
    JOIN S S3 ON SC2.学号 = S3.学号
    WHERE S3.姓名 = '张三'
  )
);

```

• 语义解读：每位学生的 MIN(成绩) > 张三的 MAX(成绩)，即“该生最低分都比张三最高分高”——等价于“该生每门课都 > 张三每门课”

• 陷阱提示：原题中“IN + > ALL”是错误写法，IN 是存在性语义，只要求“至少一门课 > ALL(张三成绩)”，并非“每门课都 >”。正确改法为 MIN/MAX (推荐) 或 NOT EXISTS 双重否定。

第 8 题：

- (1) 相关子查询 (“每门课都高于该课均分”，用 NOT EXISTS 表达“不存在一门课成绩 ≤ 该课均分”) .

```

SELECT 学号 FROM S
WHERE NOT EXISTS (
  SELECT 1 FROM SC SC1
  WHERE SC1.学号 = S.学号
  AND SC1.成绩 <= (
    SELECT AVG(SC2.成绩) FROM SC SC2
    WHERE SC2.课程号 = SC1.课程号
  )
);

```

- (2) NOT EXISTS 相关 (“没有挂过任何课” = “不存在一门课成绩 < 60”) .

```

SELECT 姓名 FROM S
WHERE NOT EXISTS (
  SELECT 1 FROM SC
  WHERE SC.学号 = S.学号 AND SC.成绩 < 60
);

```

第 9 题：

- (1)



```
SELECT C.姓名, SUM(A.余额) AS 总余额
FROM C JOIN A ON C.客户号 = A.客户号
GROUP BY C.客户号, C.姓名
HAVING SUM(A.余额) > 100000
ORDER BY 总余额 DESC;
```

• (2)

```
SELECT C.姓名, C.城市, COUNT(DISTINCT A.开户行号) AS 支行数
FROM C JOIN A ON C.客户号 = A.客户号
GROUP BY C.客户号, C.姓名, C.城市
HAVING COUNT(DISTINCT A.开户行号) >= 3
ORDER BY 支行数 DESC;
```

第 10 题 :

• (1)

```
SELECT P.姓名, COUNT(DISTINCT B.航班号) AS 订票次数
FROM P JOIN B ON P.乘客号 = B.乘客号
GROUP BY P.乘客号, P.姓名
HAVING COUNT(DISTINCT B.航班号) >= 3
ORDER BY 订票次数 DESC, P.姓名 ASC;
```

• (2)

```
SELECT P.乘客号, P.姓名,
       COALESCE(SUM(B.票数 * F.票价), 0) AS 累计金额
FROM P
LEFT JOIN B ON P.乘客号 = B.乘客号
LEFT JOIN F ON B.航班号 = F.航班号
GROUP BY P.乘客号, P.姓名
ORDER BY P.乘客号;
```

